

Spring Boot CRUD Project — Complete Notes

Lecture-based notes covering the first Spring Boot CRUD project: Postman → Controller → Service → Repository → Database, Spring Data JPA, Entity mapping, request/response JSON, HTTP methods, ResponseEntity, and project structure.

The lecture introduces CRUD as the foundation of backend applications and explains the simplified architecture before going deeper into database interaction in later lectures.

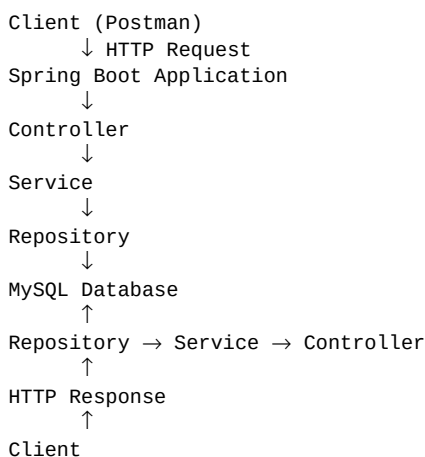
1. What Are We Building?

The project connects a Spring Boot application with a relational database and performs CRUD operations.

- Create — store a new student.
- Read — fetch one or all students.
- Update — modify an existing student.
- Delete — remove a student.
- Soft Delete — mark a record as deleted instead of physically removing it.

The lecture emphasizes that CRUD is a basic backbone of backend applications because real applications repeatedly create, read, update, and delete data.

2. Overall Application Architecture



This is the simplified Controller–Service–Repository architecture presented in the lecture.

3. Client, Server and Database

- Client: Postman is used in this project instead of building a frontend.
- Server: the Spring Boot application receives HTTP requests.
- Database: MySQL stores the student records.
- Postman sends requests; Spring Boot processes them; the database stores/retrieves data; Spring Boot returns an HTTP response.

4. Postman

Postman is used to mimic a client and send HTTP requests to the Spring Boot server.

- GET
- POST
- PUT

- PATCH
- DELETE
- Other HTTP methods exist, but the lecture focuses mainly on the methods above.

For local development, the lecture uses localhost with Spring Boot's default port 8080.

5. HTTP Methods vs CRUD Operations

HTTP Method	CRUD meaning	Typical purpose
POST	Create	Create a new resource
GET	Read	Fetch one or more resources
PUT	Update	Update an existing resource
DELETE	Delete	Delete a resource
PATCH	Partial update / soft-delete use case	Modify part of a resource

The lecture's main CRUD mapping is POST → Create, GET → Read, PUT → Update, DELETE → Delete.

6. Request Anatomy

An HTTP request can contain several important parts:

- HTTP method
- Host/base URL
- Endpoint/path
- Request body
- Headers

```
POST http://localhost:8080/api/students/create
```

Headers:

```
Accept: application/json
```

```
Content-Type: application/json
```

Body:

```
{
  "name": "Aditya",
  "email": "aditya@gmail.com",
  "age": 28,
  "rollNumber": 1001,
  "subject": "Spring Framework"
}
```

The lecture explains that Accept indicates the response format the client accepts, while Content-Type indicates the format of the request body.

7. Endpoint Concept

An endpoint can be understood as a complete URL/path mapped to a particular controller method. In the lecture, a common base path is used for student operations.

```
Base path:
/api/students
```

A method plus endpoint combination uniquely identifies which controller method should execute.

8. CRUD Endpoint Design

Create:

```
POST /api/students/create
```

Read one:

```

GET    /api/students/{id}

Read all:
GET    /api/students

Update:
PUT    /api/students/{id}

Delete:
DELETE /api/students/{id}

Soft delete:
PATCH /api/students/soft-delete/{id}

```

The lecture also demonstrates that the same path can be used by different HTTP methods because the HTTP method is part of the mapping signature.

9. Soft Delete vs Hard Delete

Hard delete physically removes the row from the database.

Soft delete keeps the row but changes a flag such as `isDeleted` from 0 to 1. Future queries can exclude records marked as deleted.

```

is_deleted = 0 → active
is_deleted = 1 → marked as deleted

```

10. Required Tools / Dependencies

- Postman — API/client testing.
- MySQL Server — relational database.
- DBeaver or MySQL Workbench — graphical database interface.
- Spring Boot Web — web/server support and embedded Tomcat.
- MySQL Driver — database connectivity.
- Spring Data JPA — database interaction through JPA abstractions.

The lecture recommends DBeaver as a beginner-friendly GUI and also mentions MySQL Workbench and other database GUI options.

11. Spring Initializr Dependencies

```

Spring Web
Spring Data JPA
MySQL Driver

```

Spring Web provides the web application infrastructure. The MySQL driver allows the application to connect to MySQL. Spring Data JPA provides the JPA-based data-access abstraction used in the project.

12. Why the Application Initially Fails

After adding database-related dependencies, the application attempts to auto-configure a `DataSource`. If database connection properties have not yet been supplied, the application can fail with a `DataSource` configuration error.

The lecture explains that database connection values belong in `application.properties`, including values such as the database URL, username, and password.

```

# Conceptual configuration
spring.datasource.url=...
spring.datasource.username=...
spring.datasource.password=...

```

The lecture temporarily demonstrates excluding `DataSource` auto-configuration so the application can start without configuring the database. This is a demonstration step; a real CRUD application needs proper database configuration.

13. Project Package Structure

```
src/main/java
├── in.dstrikes.crud.springbootdemo
│   ├── CrudSpringBootApplication.java
│   ├── controller
│   │   └── StudentController.java
│   ├── service
│   │   └── StudentService.java
│   ├── repository
│   │   └── StudentRepository.java
│   └── entity
│       └── Student.java
```

The lecture recommends separating responsibilities into packages rather than placing every class in the main package.

14. Entity — Student.java

The Student class represents the student information that the application wants to persist.

```
@Entity
public class Student {

    private Integer id;
    private String name;
    private String email;
    private Integer age;
    private Integer rollNumber;
    private String subject;

    // getters and setters
}
```

The lecture explains that @Entity tells the persistence layer that this Java class represents data that should be stored in the database.

Important: the lecture explicitly distinguishes an Entity from an IoC component. @Entity itself does not make Student a Spring IoC bean.

15. Primary Key

The Student entity contains an ID field used as the primary key.

```
@Id
private Integer id;
```

The lecture demonstrates that the database can auto-generate the ID, so the client does not necessarily need to provide it when creating a new record.

16. JSON → Java Object → Database Row

```
JSON request
  ↓
Student object
  ↓
Entity / persistence mapping
  ↓
Database table row
```

The lecture explains that the JSON request received by the controller is mapped to a Student Java object. The persistence layer then maps that object to a database row.

17. Object-Relational Mapping (ORM)

ORM means Object-Relational Mapping. The lecture introduces it as the mapping between Java objects and relational database rows.

```
Student s1 → Row 1
Student s2 → Row 2
Student s3 → Row 3
```

The lecture defers the internal details of JDBC, Hibernate, ORM, and JPA to later lessons and treats Spring Data JPA as a higher-level abstraction for this first project.

18. Controller Layer

The controller is responsible for receiving incoming HTTP requests and returning HTTP responses.

```
@RestController
public class StudentController {

    // endpoint methods
}
```

`@RestController` is a specialized controller annotation. The lecture explains its relationship to `@Controller` and ultimately `@Component`, meaning the controller is managed as a Spring bean.

19. Class-Level Request Mapping

```
@RestController
@RequestMapping("/api/students")
public class StudentController {

    ...
}
```

The common path is placed at the class level, while individual methods add their specific mapping.

20. Method-Level Mappings

```
@PostMapping("/create")
public ... createStudent(...) { }

@GetMapping("/{id}")
public ... getStudent(...) { }

@GetMapping
public ... getAllStudents() { }

@PutMapping("/{id}")
public ... updateStudent(...) { }

@DeleteMapping("/{id}")
public ... deleteStudent(...) { }
```

The lecture also discusses using different suffixes such as `/create`, `/update`, or `/delete`. It prefers keeping the HTTP method + endpoint combination unique without unnecessarily long URLs.

21. @RequestBody — JSON to Java

```
@PostMapping("/create")
public ... createStudent(
    @RequestBody Student student) {

    ...
}
```

`@RequestBody` tells Spring to take the HTTP request body and map it into the Java object. The lecture explains that Jackson performs the JSON-to-Java object conversion.

22. Jackson — JSON Conversion

- Incoming JSON → Java object when `@RequestBody` is used.
- Java object → JSON when an object is returned from the controller.
- The developer does not manually map every JSON field in the basic case.

23. Controller → Service

```
private final StudentService studentService;
```

```

public StudentController(StudentService studentService) {
    this.studentService = studentService;
}

...
Student createdStudent =
    studentService.createStudent(student);

```

The lecture prefers constructor injection instead of field injection with `@Autowired`.

24. Service Layer

The service layer contains business logic. In a simple CRUD example, there may be little or no business logic, but the service layer becomes important when validation, rules, transformations, or other application logic are needed.

```

@Service
public class StudentService {

    private final StudentRepository studentRepository;

    public StudentService(StudentRepository studentRepository) {
        this.studentRepository = studentRepository;
    }

    public Student createStudent(Student studentRequest) {
        return studentRepository.saveStudent(studentRequest);
    }
}

```

`@Service` is a specialized stereotype. The lecture explains that it is internally associated with `@Component`, while communicating the class's role more clearly to developers.

25. Repository Layer

The repository layer is responsible for interaction with the database.

```

@Component
public class StudentRepository {

    public Student saveStudent(Student student) {
        // database interaction
        return student;
    }
}

```

The lecture later introduces Spring Data JPA as the mechanism that can perform persistence operations without manually writing INSERT/SELECT/UPDATE/DELETE SQL for each basic operation.

26. Why Not Put Everything in One Class?

The lecture initially considers putting endpoint handling, business logic, and database interaction in `StudentService`, but rejects that as a scalable architecture.

- Controller → HTTP responsibility.
- Service → business responsibility.
- Repository → database responsibility.
- Entity → persistent data representation.

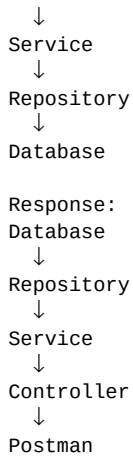
This separation follows the idea that a class should have a focused responsibility and delegate other work through dependencies.

27. Dependency Flow

```

Request:
Postman
↓
Controller

```



28. CRUD Request Flow — Create Example

1. Postman sends POST request.
2. Controller receives request.
3. @RequestBody converts JSON → Student.
4. Controller calls StudentService.
5. Service applies business logic.
6. Service calls StudentRepository.
7. Repository persists Student.
8. Result returns Repository → Service → Controller.
9. Controller returns HTTP response.

The lecture implements this flow first with dummy repository behavior before introducing actual database persistence, to keep the first project step manageable.

29. Returning a Java Object as JSON

If a controller returns a Student object from a REST endpoint, Spring/Jackson can serialize that Java object into JSON for the HTTP response.

```
{
  "id": 1,
  "name": "Aditya",
  "email": "aditya@gmail.com",
  "age": 28,
  "rollNumber": 1001,
  "subject": "Spring Framework"
}
```

30. HTTP Response Status Codes

The lecture explains that a controller response has a body and a status code. If no specific status is supplied, the basic response demonstrated uses 200 OK.

- 200 OK → successful request.
- 201 Created → a new resource was created.
- 202 Accepted → request accepted for processing.
- 404 Not Found → requested resource was not found.
- 401 Unauthorized → authentication/authorization-related failure.

31. ResponseEntity

ResponseEntity is introduced when the controller needs explicit control over the HTTP response, including the status code and response body.

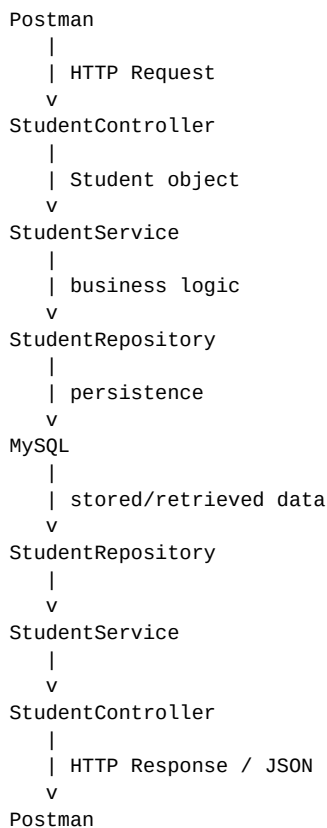
```
return ResponseEntity
    .status(HttpStatus.CREATED)
    .body(createdStudent);
```

The lecture also shows the convenient form `ResponseEntity.ok(...)` for a 200 OK response.

32. Why 201 Created for Create?

For a successful create operation, the lecture demonstrates returning HTTP 201 Created rather than the default 200 OK, because a new resource has been created.

33. Complete Mental Model



34. Key Annotations to Remember

@Entity	Marks a class as a persistence entity.
@Id	Marks the entity's primary-key field.
@RestController	Marks a REST controller managed by Spring.
@RequestMapping	Defines a common/base request path.
@PostMapping	Maps POST requests.
@GetMapping	Maps GET requests.
@PutMapping	Maps PUT requests.
@DeleteMapping	Maps DELETE requests.
@RequestBody	Maps request-body JSON to a Java object.
@Service	Marks a service-layer Spring bean.
@Component	Generic Spring component stereotype.

35. Important Distinctions

- Entity $\neq$ IoC component: `@Entity` is for persistence mapping; it does not by itself make the class a Spring bean.
- Controller $\neq$ Service: controller handles HTTP; service handles business logic.
- Service $\neq$ Repository: service coordinates business logic; repository handles persistence.

- HTTP method $\neq$ CRUD operation: POST/GET/PUT/DELETE are HTTP methods; Create/Read/Update/Delete describe the application operations.
- Hard delete $\neq$ soft delete: hard delete removes the row; soft delete marks it as deleted.
- JPA $\neq$ SQL: JPA provides an abstraction for persistence; the lecture postpones the underlying SQL/JDBC/Hibernate internals.

36. Interview Questions

Why do we use Controller-Service-Repository architecture?

To separate HTTP handling, business logic, and database interaction into focused responsibilities.

What is @RequestBody?

It tells Spring to map the HTTP request body into a Java object; the lecture attributes the JSON conversion to Jackson.

Why use constructor injection?

It makes dependencies explicit and is preferred in the lecture over field injection.

What is an Entity?

A Java class representing persistent data that can be mapped to a database table.

Does @Entity make a class a Spring bean?

Not according to the lecture; persistence/JPA manages the entity, not the Spring IoC container as a normal component.

What is ORM?

Object-Relational Mapping: mapping Java objects to relational database rows.

Why use Spring Data JPA?

It allows common database operations without manually writing SQL queries for each basic CRUD operation.

Why use ResponseEntity?

To explicitly control the HTTP response, including status and body.

Why return 201 for create?

Because a new resource has been created.

What does @RestController indicate?

That the class handles REST requests and produces HTTP responses, commonly JSON.

37. Quick Revision

- CRUD = Create, Read, Update, Delete.
- Postman acts as the client.
- Spring Boot acts as the server/application.
- MySQL stores the data.
- Controller receives HTTP requests.
- Service contains business logic.
- Repository communicates with the database.
- Entity represents persistent data.
- @RequestBody maps JSON $\rightarrow$ Java object.
- Jackson maps Java object $\rightarrow$ JSON response.

- Spring Data JPA reduces manual SQL for common persistence operations.
- ResponseEntity gives explicit response-status/body control.
- Use constructor injection for dependencies.
- Keep packages separated by responsibility.

38. What This Lecture Completes

This lecture builds the conceptual and initial code structure for the CRUD application and implements the create-request flow up to a dummy repository response. The lecture explicitly leaves actual database interaction and the remaining Read, Update, and Delete implementations for the next video.

The purpose is to establish the architecture first so that the later database/JPA implementation is easier to understand.